

# Axiomander Roadmap: From Contract Checking to Specification Management

## Vision

The long-term goal of Axiomander is not merely to verify code.

The goal is to make **specifications the primary artifact** and implementations a secondary concern.

In such a system:

- Contracts describe intended behaviour.
- Implementations satisfy contracts.
- Verification establishes consistency between contracts and implementations.
- AI agents operate primarily on specifications.
- Human developers manage specifications rather than individual proofs.

This is the transition from:

Code-Centric Development

to:

Specification-Centric Development

or more simply:

Code as an implementation detail.  
Specifications as the source of truth.

---

## Current Position

Axiomander already possesses many of the foundational components:

- Preconditions
- Postconditions
- Invariants
- Old-state references

- Frame conditions
- Exception contracts
- Units and dimensions
- Pydantic integration
- SMT verification
- Coq verification
- Proof caching
- Dependency tracking
- MCP tooling

The challenge is no longer proving simple numerical properties.

The challenge is managing specifications for realistic software systems.

---

## Guiding Principle

The next phase should not focus primarily on:

```
More WP generation.  
More SMT tricks.  
More syntax.
```

Instead it should focus on:

```
Specification management.  
Proof management.  
Resource modelling.  
System-wide reasoning.
```

---

## Strategic Architecture

### Layer 1: Specification Graph

#### Objective

Create a persistent graph representing all specifications in the codebase.

Every function, type, model, and proof obligation should become a node.

## Example

```
FunctionSpec
├ requires
├ ensures
├ raises
├ reads
├ modifies
├ units
├ shape obligations
├ dependencies
├ proof obligations
└ proof status
```

## Benefits

Allows the system to answer:

```
What changed?

Which proofs are invalidated?

Which callers are affected?

Which assumptions remain unproved?

Where are trusted axioms used?
```

## Recommendation

Introduce:

```
SpecificationGraph
ProofGraph
DependencyGraph
```

as first-class persistent artefacts.

SQLite is sufficient initially.

---

# Layer 2: Proof Provenance

## Objective

Every proof obligation must record:

- how it was proved
- what assumptions were used
- when it was last validated

Verification should never simply produce:

Verified

Instead:

Verified by what?  
Verified under which assumptions?  
Verified using which trust boundary?

## Proposed Status Model

PROVED\_L1\_COQ  
  
PROVED\_L2\_SMT  
  
PROVED\_L2B\_THEORY  
  
PROVED\_L3\_ORACLE  
  
ASSUMED\_STUB  
  
USER\_AXIOM  
  
COUNTEREXAMPLE\_FOUND  
  
STALE  
  
UNKNOWN

## Benefits

Creates auditability.

Allows organisations to define policies such as:

Production code may not depend on ORACLE proofs.

---

## Layer 3: Resource and Region Logic

### Objective

Move beyond local variable frame conditions.

Current frame conditions are useful but insufficient for realistic software.

We need abstract resources.

---

### Proposed Regions

```
heap:user  
  
heap:session  
  
db:orders  
  
db:customers  
  
queue:email_jobs  
  
queue:billing_jobs  
  
fs:reports  
  
fs:cache  
  
api:stripe  
  
api:openai
```

## Example

```
"""
reads:
    db:orders[order_id]

modifies:
    queue:email_jobs

ensures:
    email_job_enqueued(order_id)
"""
```

---

## Why This Matters

This is the bridge from:

Reasoning about Python values

to:

Reasoning about software systems

including:

- databases
- queues
- APIs
- caches
- filesystems
- distributed services

---

## Layer 4: Domain Models via Pydantic

### Objective

Treat Pydantic as a specification language.

Not merely as runtime validation.

---

## Example

```
class Account(BaseModel):  
    id: AccountId  
    balance: Money  
    status: Literal[  
        "open",  
        "frozen",  
        "closed"  
    ]
```

---

## Automatic Extraction

Generate:

```
Shape constraints  
  
Field constraints  
  
Construction invariants  
  
Serialization invariants  
  
Database mappings
```

---

## Long-Term Goal

Contracts should be written primarily against domain models.

Not raw dictionaries.

Not arbitrary objects.

Not implementation details.

---

# Layer 5: Refinement Levels

## Objective

Provide a gradual adoption path.

Most Python systems cannot begin at full verification.

---

## Proposed Levels

### Level 0

Type checking

Shape checking

Linting

---

### Level 1

Local contracts

Pure reasoning

No heap

---

### Level 2

Frames

Old-state references

Exception specifications

---

### Level 3

Pydantic domain models

Object invariants

Shape reasoning

---

#### **Level 4**

Resource regions

Databases

Queues

Filesystems

External services

---

#### **Level 5**

Concurrency protocols

Ownership

Shared resources

Locks

---

#### **Level 6**

End-to-end workflow specifications

System-wide guarantees

Business process verification

---

## **Most Important Missing Features**

The following gaps currently prevent Axiomander from becoming a top-tier specification platform.

### **1. Persistent Specification Graph**

Highest priority.

Everything else builds upon it.

---

## 2. Proof Provenance

Must know:

What is proved?  
What is assumed?

at all times.

---

## 3. Region-Based Frames

Required for:

- databases
  - queues
  - files
  - APIs
- 

## 4. Effect Adapters

Create abstract specifications for:

SQL  
  
Redis  
  
Filesystem  
  
HTTP  
  
Message queues

These become logical resources.

---

## 5. Protocol Specifications

Need state-machine contracts.

Example:

```
Order

Created
→ Paid
→ Fulfilled
→ Archived
```

Verification should reason about legal transitions.

---

## 6. Concurrency Contracts

Need support for:

```
Ownership

Resource transfer

Queue consumption

Lock invariants

Happens-before relationships
```

This will eventually require separation-logic-style concepts.

---

## 7. Counterexample-Guided Refinement

When proofs fail:

```
Generate counterexample.

Locate weakest assumption.

Suggest contract repair.
```

AI assistance becomes dramatically more effective.

---

## 8. Specification Dashboard

A human-facing view of:

Proof status  
Trust boundaries  
Assumptions  
Dependencies  
Coverage

for the entire system.

---

## Recommended Development Order

### Phase 1

Specification Graph

Proof Provenance

Dependency Persistence

---

### Phase 2

Region Logic

Effect Adapters

Database Resources

Queue Resources

Filesystem Resources

---

## Phase 3

Pydantic Domain Extraction

Automatic Invariants

State-Machine Specifications

---

## Phase 4

Concurrency Model

Ownership

Resource Transfer

Protocol Verification

---

## Phase 5

AI Specification Management

Contract suggestion

Contract repair

Assumption reduction

Proof guidance

Specification evolution

---

## End State

The end state is not a stronger assertion checker.

The end state is a system where:

Specifications define reality.

Implementations satisfy specifications.

Verification maintains consistency.

AI agents operate primarily on specifications.

At that point Axiomander becomes:

A specification operating system for Python.

rather than merely:

A contract verification framework.